

## Week 7: User Authentication and Session Management

### *Practical Task 1 — Student Portal Login System*

|                                      |                                                                                            |
|--------------------------------------|--------------------------------------------------------------------------------------------|
| <b>Student Name</b>                  | Stephen Otiende                                                                            |
| <b>Admission Number</b>              | BBIT/2018/36596                                                                            |
| <b>Unit Code / Name</b>              | BIT3208 — Advanced Web Design and Development                                              |
| <b>Lecturer</b>                      | Mr. Michael Nyoro                                                                          |
| <b>Project Title</b>                 | Student Portal Login System (Standalone Practical Task)                                    |
| <b>Technologies (Week 7)</b>         | PHP (mysqli), MySQL, HTML5, CSS3, PHP Sessions                                             |
| <b>Week Topic (per Weekly Brief)</b> | User Authentication and Session Management — Practical Task 1: Student Portal Login System |

### Title

Student Portal Login System — Registration, Authentication, Dashboard, and Logout

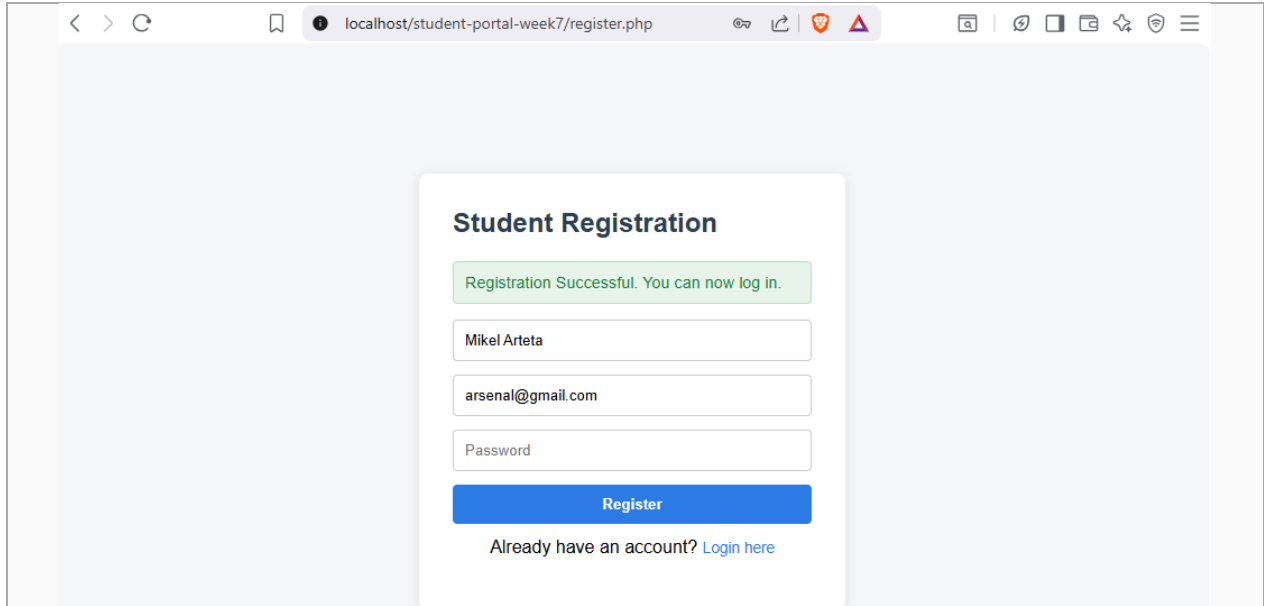
### What Was Built

A standalone Student Portal Login System was built directly from this week's practical brief, independent of any other project. The system covers the full authentication workflow taught this week: a registration form that hashes passwords with `password_hash()` before storing them in MySQL, a login form that verifies credentials with `password_verify()` and starts a PHP session on success, a protected dashboard that checks for an active session before granting access, and a logout script that destroys the session. The users table stores fullname, email, and password (hashed), matching the structure given in the weekly material. `mysqli` is used throughout for the database connection, consistent with the brief's `connection.php` example.

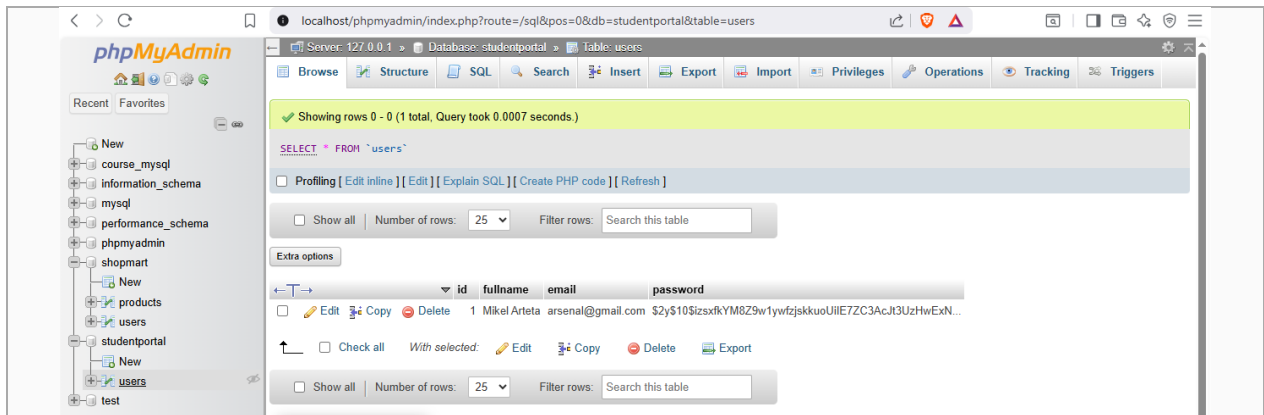
### Key Features Implemented

- `index.html` — landing page linking to Login and Register.
- `register.php` — registration form with input validation, duplicate-email check, and `password_hash()` before insert.
- `login.php` — verifies credentials with `password_verify()` and creates `$_SESSION['user']` on success.
- `dashboard.php` — protected page; redirects to `login.php` if no session exists.
- `logout.php` — destroys the session and redirects to login.
- users table (MySQL) — id, fullname, email, password (hashed), matching the brief's schema.

### Required Evidence

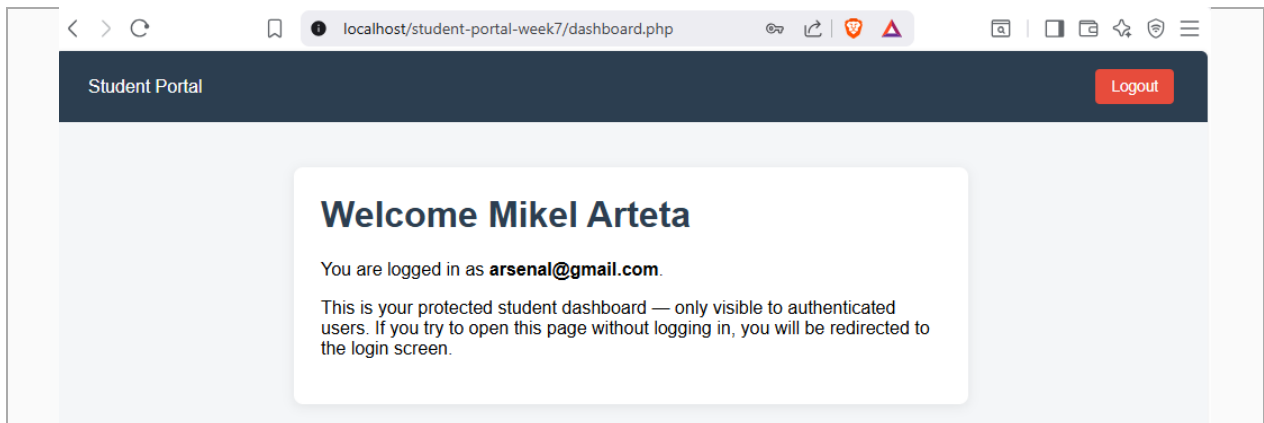


**Fig 1: Registration Form & Success Message** — *register.php* showing a filled-in registration form and the “Registration Successful” message after submission.

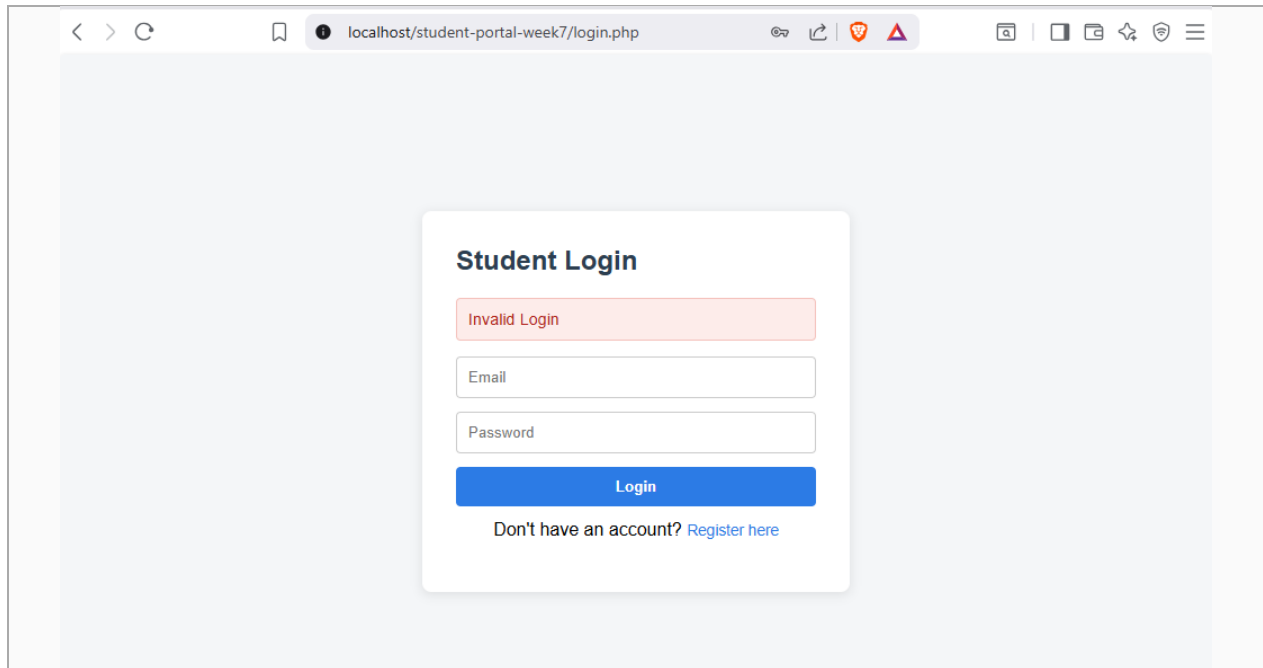


| id | fullname     | email             | password                                                 |
|----|--------------|-------------------|----------------------------------------------------------|
| 1  | Mikel Arteta | arsenal@gmail.com | \$2y\$10\$izsxRkYM8Z9w1ywfzjskkuoUIIE7ZC3AcJt3UzHwExN... |

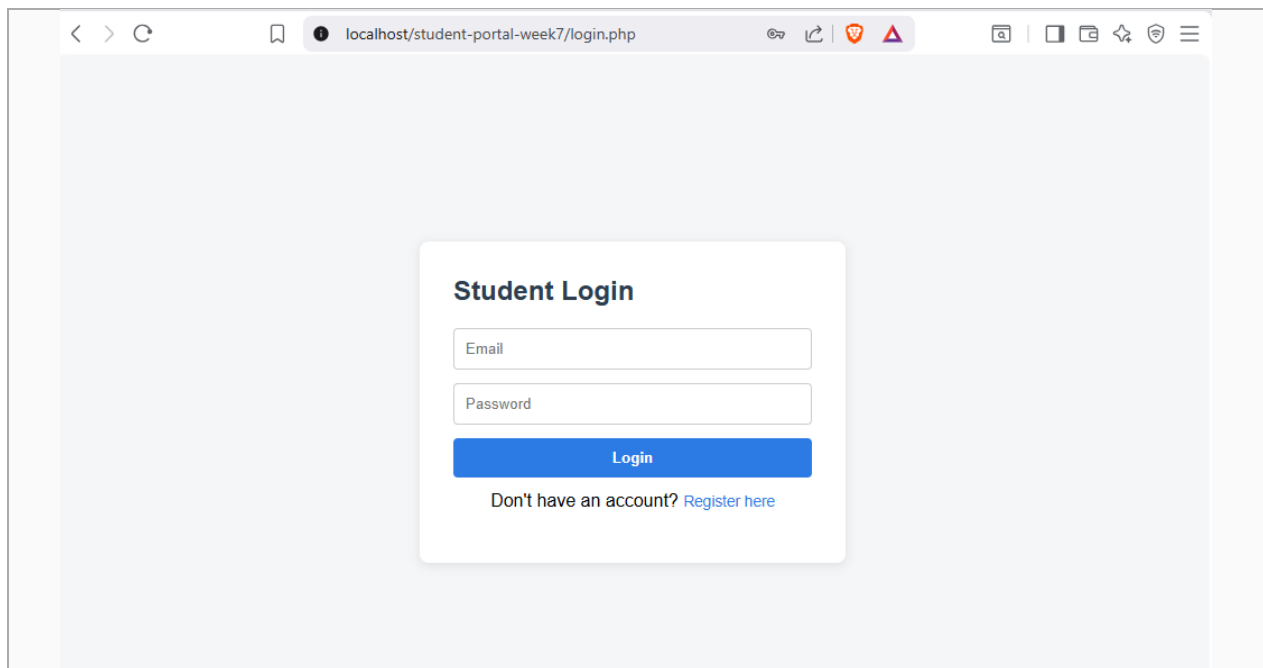
**Fig 2: Hashed Password in Database** — *phpMyAdmin* view of the *users* table showing the password column stored as a bcrypt hash (e.g. starting with `$2y$10$`), never as plain text.



**Fig 3: Successful Login & Dashboard** — *dashboard.php* after logging in, showing “Welcome [Full Name]” pulled from `$_SESSION['user']`.



**Fig 4: Invalid Login Attempt** — *login.php* showing the “Invalid Login” message when incorrect credentials are submitted.



**Fig 5: Protected Page Redirect & Logout** — Browser address bar showing *dashboard.php* redirecting to *login.php* when accessed while logged out, and the result of clicking Logout.

## Weekly Reflection Questions

**What is authentication?** Authentication is the process of verifying that a user is who they claim to be, typically by checking a submitted email/username and password against stored records.

**How does authorization differ from authentication?** Authentication confirms identity; authorization determines what an already-identified user is allowed to do or access.

**Why should passwords be hashed?** Hashing converts a password into an irreversible string before storage, so that even if the database is leaked, the original passwords are not exposed in plain text.

**What is the purpose of sessions?** Sessions let the server remember a logged-in user across multiple page requests, since HTTP itself does not retain state between requests.

**Why are protected pages important?** They ensure that only authenticated users can view or act on sensitive content, preventing unauthorized access to pages like the dashboard.

**What are the dangers of SQL injection?** An attacker can manipulate a query (e.g. entering ' OR '1'='1) to bypass authentication or extract/alter data; prepared statements prevent this by separating SQL code from user input.

**How does logout improve security?** Logout destroys the session immediately, so the session ID can no longer be used to access protected pages, reducing the window for session hijacking.

### Student Reflection (Max 100 Words)

This week made authentication concrete rather than theoretical. Building registration, login, and logout independently — without any other project around it — made it easier to focus purely on the security mechanics: hashing, password\_verify(), and session checks. The trickiest part was remembering that the dashboard must check the session on every load, not just at login, otherwise a logged-out user could still reach it by typing the URL directly. This matters because authentication is the foundation every other feature in a web app depends on — without it, there is no way to know who is making a request.

### Tools Used

- XAMPP — local Apache web server and MySQL database
- phpMyAdmin — verifying the users table and confirming passwords are hashed
- VS Code — writing the PHP authentication scripts